MATERIAL DIDÁCTICO · PMDM

UNIDAD DIDÁCTICA 02

Objetos, colecciones y listas en Android

APRENDER PROGRAMANDO, PASO A PASO

PROGRAMACIÓN MULTIMEDIA Y DISPOSITIVOS MÓVILES · 2.º DAM

KOTLIN INTERMEDIO · POO · LISTVIEW · RECYCLERVIEW

Guía de trabajo

Partimos de los formularios y cálculos de la UD1. Ahora una aplicación dejará de manejar un solo dato: trabajaremos con muchos elementos, representaremos entidades mediante clases y aprenderemos a mostrarlos en listas Android.

Bloque	Qué aprenderás
1 · Kotlin con varios datos	for, rangos, listas, operaciones y búsquedas
2 · Modelos y clases	Constructores, propiedades, métodos, data class y encapsulación
3 · Interfaz con listas	ListView, ArrayAdapter y eventos sobre elementos
4 · Listas avanzadas	RecyclerView, ViewHolder, Adapter y actualización
5 · Prácticas	Tutorial 1 → Relación 1 → Tutorial 2 → Relación 2

> **Reto de la unidad:** construir un inventario Android capaz de añadir productos y mostrar sus cantidades. Los datos estarán en memoria: se perderán al cerrar el proceso; aprenderemos a conservarlos en la UD3.

Antes de comenzar

- Android Studio preparado, un emulador o teléfono disponible y una plantilla **Empty Views Activity** con Kotlin.
- Conocer **EditText, TextView, Button**, **findViewById**, eventos **setOnClickListener** y conversiones seguras.
- Continuaremos usando XML (Views); no cambiaremos a Jetpack Compose a mitad de la secuencia.

1. Colecciones y bucles de Kotlin

1.1 De una variable a una lista

Una variable permite guardar un valor. Una **lista** agrupa varios valores ordenados; cada posición empieza en 0. Una lista de solo lectura se crea con **listOf**; una mutable permite añadir, eliminar o sustituir elementos.

KOTLIN / CÓDIGO COMPLETO O FRAGMENTO

```
val fijas = listOf("Teclado", "Ratón", "Monitor")
val materiales = mutableListOf("Cable", "Proyector")
materiales.add("Altavoz")
materiales.remove("Cable")
val primero = materiales[0]  // Proyector
val total = materiales.size   // 2
```

Que una lista sea de solo lectura no significa que sus objetos internos sean inmutables. **MutableList** sí permite operaciones como add, removeAt y clear. Si consultas una posición inexistente se produce un error: verifica los límites antes de usar índices.

1.2 Recorre los elementos con for

KOTLIN / CÓDIGO COMPLETO O FRAGMENTO

```
val precios = listOf(2.5, 8.0, 1.5)
var total = 0.0
for (precio in precios) {
    total += precio
}
val indices = precios.indices
for (i in indices) {
    val elemento = precios[i]
}
```

for repite un bloque por cada elemento. **indices** ofrece las posiciones válidas y evita escribir un rango incorrecto. Los rangos también se usan en validaciones: **edad in 18..65** comprueba un entero entre ambos extremos.

1.3 Operaciones expresivas

KOTLIN / CÓDIGO COMPLETO O FRAGMENTO

```
val temperaturas = listOf(18, 24, 31, 17)
val altas = temperaturas.filter { it >= 25 }  // [31]
val dobles = temperaturas.map { it * 2 }
val maximo = temperaturas.maxOrNull()        // Int?
val suma = temperaturas.sum()                // 90
val existe = temperaturas.any { it < 18 }    // true
```

filter selecciona elementos, **map** transforma cada uno, **sum** suma y **any** comprueba si existe al menos una coincidencia. **it** representa el elemento actual. No confundas **filter** con modificar la colección: devuelve una lista nueva.

> **Comprueba:** antes de ejecutar, predice qué obtendrás con temperaturas.filter { it >= 18 }. Prueba también una lista vacía: maxOrNull() devuelve null.

2. Clases, objetos y modelos de datos

2.1 Diseñar un objeto de tu aplicación

Un objeto reúne datos relacionados y comportamientos. Una clase define qué propiedades y métodos tendrán sus objetos. El constructor principal recoge los datos obligatorios al crearlos.

KOTLIN / CÓDIGO COMPLETO O FRAGMENTO

```
class Reserva(val cliente: String, var plazas: Int) {
    fun puedeEntrar(aforo: Int): Boolean {
        return plazas > 0 && plazas <= aforo
    }
}

val reserva = Reserva("Ana", 3)
val valida = reserva.puedeEntrar(10)
```

En el constructor, **val** impide reasignar esa propiedad y **var** permite cambiarla. Un método calcula o devuelve un valor; la Activity decide cómo mostrarlo. No uses **println** como mecanismo de salida de métodos de dominio.

2.2 data class: una ficha sencilla

KOTLIN / CÓDIGO COMPLETO O FRAGMENTO

```
data class Libro(
    val isbn: String,
    val titulo: String,
    val paginas: Int
)

val libro = Libro("978-000", "La isla", 240)
val copia = libro.copy(paginas = 250)
val nombre = libro.titulo
```

Una **data class** es cómoda para entidades cuyo objetivo principal es guardar datos. Kotlin genera funciones útiles como **toString**, **equals** y **copy**. **copy** genera otro objeto, no modifica el original.

2.3 Encapsulación y validaciones

KOTLIN / CÓDIGO COMPLETO O FRAGMENTO

```
class Cuenta(private var saldo: Double = 0.0) {
    fun ingresar(cantidad: Double): Boolean {
        if (cantidad <= 0.0) return false
        saldo += cantidad
        return true
    }

    fun consultarSaldo(): Double = saldo
}
```

Con **private** evitamos que otras clases modifiquen directamente el saldo. **ingresar** informa si la operación salió bien. En la pantalla validaremos el texto antes de convertirlo; la clase comprobará además sus propias reglas de negocio.

2.4 Herencia e interfaces: reconocer el patrón

KOTLIN / CÓDIGO COMPLETO O FRAGMENTO

```
open class Elemento(val nombre: String) {
    open fun descripcion(): String = nombre
}
class Material(nombre: String, val unidades: Int) :
    Elemento(nombre) {
    override fun descripcion(): String {
        return "$nombre · $unidades unidades"
    }
}
```

La herencia reutiliza o especializa un tipo. En Kotlin una clase es final por defecto; **open** permite heredarla y **override** redefinir un comportamiento. No abuses de la herencia: para fichas simples del catálogo suele bastar una data class.

3. Kotlin seguro para aplicaciones

3.1 Nulos, conversiones y operadores útiles

KOTLIN / CÓDIGO COMPLETO O FRAGMENTO

```
val numero: Int? = "15".toIntOrNull()
val seguro = numero ?: 0
val longitud = numero?.toString()?.length
val ciudad = "  Granada  ".trim()
val nombreValido = ciudad.isNotBlank()
```

?: ofrece un valor alternativo si hay null; **?.** ejecuta una operación solo cuando el valor no es nulo. Evita **!!** en entradas del usuario: forzaría una excepción si el dato no existe.

3.2 Buscar, añadir sin repetir y calcular

KOTLIN / CÓDIGO COMPLETO O FRAGMENTO

```
data class Tarea(val titulo: String, val minutos: Int)

fun buscarTarea(datos: List<Tarea>, nombre: String): Tarea? {
    return datos.find { it.titulo.equals(nombre, true) }
}

fun totalMinutos(datos: List<Tarea>): Int {
    return datos.sumOf { it.minutos }
}
```

El método **equals(nombre, true)** permite comparar texto sin distinguir mayúsculas. **find** devuelve un elemento o null si no existe. **sumOf** agrega la propiedad indicada.

> **Separación que mantendremos:** los métodos de cálculo y búsqueda reciben datos y devuelven resultados. La Activity se encarga de leer EditText, actualizar TextView y escuchar botones.

4. Mostrar listas en Android con ListView

4.1 Datos, adaptador y vista

Un **ListView** muestra una colección mediante un **ArrayAdapter**. La colección contiene los datos; el adaptador los transforma en filas visibles. Tras una modificación, avisa con **notifyDataSetChanged()**.

KOTLIN / CÓDIGO COMPLETO O FRAGMENTO

```
val datos = mutableListOf("Uno", "Dos")
val adaptador = ArrayAdapter(
    this, android.R.layout.simple_list_item_1, datos
)
val lista = findViewById<ListView>(R.id.lvDatos)
lista.adapter = adaptador

datos.add("Tres")
adaptador.notifyDataSetChanged()
```

El patrón sirve para listas sencillas de texto. Para manejar una fila concreta, utiliza **setOnItemClickListener**; el parámetro posición indica qué elemento se ha pulsado.

4.2 Tutorial 1: Mi lista de compra

1. Crea el proyecto **ListaCompra** con Empty Views Activity y paquete **com.example.listacompra**.
2. Abre el PDF **UD2_Tutorial_1_Lista_de_compra.pdf** y sigue los pasos antes de realizar la Relación 1.
3. Comprueba añadir, borrar pulsando una fila, lista vacía y cadena con solo espacios.

> **Relación 1:** cinco ejercicios de colecciones, clases y ListView. Resuélvelos antes de pasar a RecyclerView.

5. Listas dinámicas con RecyclerView

5.1 Por qué necesitamos otro componente

Para filas con varios campos o listas más complejas, **RecyclerView** separa tres responsabilidades: **RecyclerView** es el contenedor, **LayoutManager** ordena las filas y **Adapter** conecta datos con las vistas. El **ViewHolder** conserva referencias a los controles de cada fila y permite reutilizarlos.

RecyclerView sigue siendo válido para proyectos Android Views, aunque su biblioteca está en mantenimiento y Google recomienda Compose para interfaces nuevas. Aquí lo usamos porque continuamos deliberadamente la ruta XML de la UD1.

5.2 Preparar la dependencia

GRADLE KOTLIN DSL / CÓDIGO COMPLETO O FRAGMENTO

```
// En app/build.gradle.kts, dentro de dependencies:
implementation("androidx.recyclerview:recyclerview:1.4.0")
```

Pulsa **Sync Now**. La biblioteca 1.4.0 requiere compilar con una API de Android suficientemente reciente (usa **compileSdk 35 o superior**). **minSdk** puede ser menor; ambas propiedades no son lo mismo.

5.3 Modelo, diseño de fila y Adapter

KOTLIN + XML (CONCEPTUAL) / CÓDIGO COMPLETO O FRAGMENTO

```
data class Producto(val nombre: String, val cantidad: Int)

// En el XML principal se declara:
<androidx.recyclerview.widget.RecyclerView
    android:id="@+id/rvProductos"
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_weight="1" />
```

Cada fila tendrá un diseño propio, por ejemplo **item_producto.xml**, con dos TextView. El Adapter crea el ViewHolder cuando se necesita una fila, y enlaza cada Producto con los TextView que correspondan.

KOTLIN / CÓDIGO COMPLETO O FRAGMENTO

```
override fun onBindViewHolder(
    holder: ProductoViewHolder, position: Int
) {
    val producto = datos[position]
    holder.nombre.text = producto.nombre
    holder.cantidad.text = "Unidades: ${producto.cantidad}"
}
override fun getItemCount(): Int = datos.size
```

Cuando añades un elemento a la lista mutable, comunica el cambio mediante **notifyItemInserted(indice)**; así la vista puede actualizar la fila indicada. No crees manualmente un TextView por cada elemento.

5.4 Tutorial 2: Inventario del aula

1. Crea **InventarioPMDM** con paquete **com.example.inventariopmdm**.
2. Añade la dependencia AndroidX y sincroniza.
3. Sigue el tutorial completo para crear **Producto.kt**, **ProductoAdapter.kt**, dos archivos XML y MainActivity.kt.
4. Añade material, comprueba unidades, datos no válidos y desplazamiento por la lista.

> **Relación 2:** cuatro proyectos con modelos propios y RecyclerView, más una ampliación final. No se pide persistencia todavía.

6. Comprobaciones, errores y entrega

6.1 Fallos que debes saber diagnosticar

Síntoma	Causa probable / qué revisar
La lista no cambia	Datos modificados sin notificar al adaptador
R.id no encuentra una vista	ID distinto entre XML y Kotlin, o proyecto sin compilar
RecyclerView queda vacío	Falta layoutManager, adapter o hay cero elementos
Fallo de Gradle	Dependencia fuera de dependencies o sincronización pendiente
NumberFormatException	Conversión con toInt() en lugar de toIntOrNull()
Los datos desaparecen al girar	Lista en memoria de Activity; estado y persistencia en UD3

6.2 Pruebas mínimas

- Alta de dos elementos correctos: aparecen en el orden esperado y el contador coincide.
- Campo vacío o formado por espacios: se muestra un aviso sin añadir nada.
- Cantidad 0, negativa, decimal y con letras: se rechaza sin cerrar la app.
- Lista con muchos elementos: desplazamiento fluido y sin filas mezcladas.
- En la entrega incluye capturas de resultado correcto y error, además del proyecto completo.

Consulta oficial: RecyclerView AndroidX. https://developer.android.com/develop/ui/views/layout/recyclerview

Consulta oficial: Versión de RecyclerView. https://developer.android.com/jetpack/androidx/releases/recyclerview

> **Al terminar la UD2** debes poder modelar entidades, mantener colecciones, mostrarlas en Android y separar la lógica de datos del código de interfaz.